

Big Data Processing Architectures and Their Role in the Future of AI Systems

1. Introduction

Over the last decades, data systems have evolved in different directions depending on the type of workload they were designed to support. On one side, we had analytical systems such as data warehouses and OLAP engines. On another side, we saw the rise of event streaming platforms, stream processors, and Change Data Capture (CDC) pipelines. Each of these systems originally solved a specific problem: reporting, real-time processing, integration, or scaling read-heavy workloads.

Today, Artificial Intelligence systems, especially Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and autonomous AI agents, are changing the way we think about data architectures. AI systems generate large amounts of data, require continuous updates, and perform a massive number of reads. This creates new pressure on data infrastructures.

In this report, I analyze how traditional analytical systems, streaming systems, and incremental computation techniques interact. I explain their differences and overlaps, and I take a position on how AI workloads will shape future data architectures.

2. Analytical Processing Foundations

2.1 Analytical vs Transactional Processing

The first important distinction in data systems is between transactional processing (OLTP) and analytical processing (OLAP).

Transactional systems are optimized for:

- High concurrency
- Small, short-lived queries
- Strong consistency
- Frequent writes and updates

Typical examples are banking systems, e-commerce checkouts, and user account management. These systems usually rely on normalized schemas and B+ tree indexes for fast point lookups.

Analytical systems, in contrast, are optimized for:

- Large-scale aggregations
- Complex queries across large datasets
- Scans and joins over millions or billions of rows
- Read-heavy workloads

Data warehouses and OLAP engines (such as columnar databases) store data in a way that makes scanning and aggregation efficient. Instead of optimizing for single-row updates, they optimize for analytical queries like:

- “What was total revenue per region over the last 12 months?”
- “Which user segments have the highest churn risk?”

This difference in workload leads to different architectural choices: columnar storage, compression, vectorized execution, and distributed query engines.

2.2 Role of Indexes in Analytical Systems

Indexes are fundamental to performance in both OLTP and OLAP systems, but their role differs.

In transactional systems, B+ tree indexes allow fast point lookups and range scans. They reduce the need to scan entire tables.

In analytical systems:

- Secondary indexes are often less central because large scans are common.
- Columnar storage itself acts like a form of indexing by allowing the engine to read only relevant columns.
- Bitmap indexes and zone maps are used to prune large parts of the dataset.

Recently, LSM-based indexes have also become important, especially in systems that handle both writes and reads at scale. LSM trees optimize for high write throughput, which becomes relevant in hybrid systems where data is continuously ingested and queried.

2.3 Materialized Views and Query Optimization

Materialized views are precomputed query results stored physically. Instead of recomputing aggregations from raw data, the system maintains a derived table.

This is critical in analytical workloads because:

- Queries can be expensive.
- Data volumes are large.
- Many users might request similar aggregations.

Query optimizers decide whether to:

- Use base tables,
- Use indexes,
- Or rewrite queries to use materialized views.

Incremental view maintenance is especially important. Instead of recomputing the entire view when data changes, only the delta is applied. This reduces computation costs and improves freshness.

Materialized views and incremental maintenance are key concepts that reappear in streaming and AI workloads.

3. Streaming, Event Processing, and CDC

3.1 From Batch to Streams

Traditional analytics relied heavily on batch processing. Data was collected during the day and processed overnight. This model is simple and stable, but not suitable for real-time use cases.

Stream processing systems (such as Spark Structured Streaming or Flink) treat data as an unbounded sequence of events. Instead of running periodic batch jobs, they continuously process new data.

Event streaming platforms like Kafka provide:

- Durable event logs
- High-throughput ingestion
- Replay capabilities

CDC pipelines capture changes from transactional databases and publish them as events. This allows analytical systems and downstream consumers to react to changes without full table scans.

3.2 Stream Processing vs Incremental Analytics

Stream processing and incremental analytics overlap conceptually.

Both:

- Process data incrementally.
- Maintain derived state.
- Avoid full recomputation.

However, there are differences.

Stream processing:

- Focuses on low-latency event handling.
- Works with event time, windows, and continuous operators.
- Often prioritizes responsiveness.

Incremental analytics in warehouses:

- Focuses on efficient maintenance of aggregated views.
- May tolerate slightly higher latency.
- Emphasizes correctness and consistency.

In practice, modern systems blur these boundaries. Some data warehouses now support streaming ingestion and near-real-time materialized views. Similarly, stream processors maintain state stores that resemble databases.

3.3 Latency vs Consistency Trade-offs

One central trade-off is latency versus consistency.

- Low latency requires immediate processing and potentially relaxed guarantees.
- Strong consistency may require coordination, checkpoints, and transactional guarantees.

In stream systems:

- Exactly-once semantics are complex and costly.

- Many systems offer at-least-once processing for performance.

In analytical systems:

- Snapshot isolation and transactional consistency are common.
- Updates may be slower but more predictable.

As systems converge, designers must balance:

- Freshness
 - Throughput
 - Consistency
 - Operational complexity
-

4. Implications for AI Systems

AI workloads change the game significantly.

4.1 High-Volume Ingestion and Continuous Updates

AI systems generate and consume data continuously:

- Logs of user interactions with LLMs
- Feedback signals for fine-tuning
- Embeddings for RAG systems
- Agent-generated events

This leads to:

- High write rates
- Frequent updates
- Continuous retraining pipelines

Traditional batch-oriented warehouses are not sufficient on their own. Incremental computation becomes central.

4.2 Amplification of Incremental Computation

AI systems amplify the importance of incremental computation for several reasons:

1. Data changes constantly.
2. Models must be updated frequently.
3. Derived artifacts (embeddings, features, summaries) must remain fresh.

For example, in RAG systems:

- New documents are embedded and added to a vector index.
- Query results must reflect recent updates.
- Full recomputation of the index is not feasible at scale.

This is essentially incremental view maintenance applied to embeddings and vector stores.

Similarly, AI agents that interact with systems generate large feedback loops:

- Each action produces new events.
- These events influence future decisions.
- The system must react quickly.

Without incremental architectures, costs would explode.

4.3 High Query Rates and Read Amplification

LLMs and AI agents often generate many internal queries:

- Retrieval queries to vector databases.
- Metadata lookups.
- Context assembly operations.

This creates massive read amplification. Systems must support:

- High concurrency
- Low latency reads
- Caching and serving layers

This shifts importance toward:

- Specialized serving layers
- In-memory indexes
- Distributed caches

In addition, freshness becomes critical. An outdated embedding index can lead to incorrect answers. Therefore, the architecture must combine:

- Fast ingestion
- Efficient incremental updates
- Scalable serving

4.4 Training and Fine-Tuning Workloads

Training LLMs or fine-tuning models involves:

- Large-scale batch processing.
- Distributed computation over huge datasets.
- High-throughput data loading.

Here, traditional data lakes and warehouses still matter. They provide:

- Reliable storage
- Data governance
- Schema management

However, the data feeding training pipelines is increasingly event-driven and continuously updated.

This creates a hybrid requirement:

- Batch systems for large-scale training.
 - Streaming systems for continuous adaptation.
 - Feature stores to bridge training and serving.
-

5. Technical Positioning and Future Outlook

5.1 Unified Architectures or Specialization?

In my view, AI workloads will push toward partially unified architectures, but not full convergence.

There is strong pressure for:

- Unified storage layers.
- Shared metadata and schema management.
- Common incremental computation frameworks.

However, complete unification is unrealistic because:

- Training workloads have very different characteristics than low-latency serving.
- Real-time streaming has different constraints than long-running analytical queries.
- Operational complexity increases if everything is forced into a single system.

Instead, I expect a layered but tightly integrated architecture:

- A streaming backbone for ingestion and change propagation.
- A scalable storage layer (lakehouse style).
- Incremental computation engines that work in both batch and streaming mode.
- Specialized serving layers for AI inference and retrieval.

5.2 Components That Become Central

The components that will become more central are:

1. Streaming platforms: As the backbone of data movement.
2. Incremental view maintenance: To avoid recomputation.
3. Serving layers: To handle high read concurrency.
4. Vector and feature stores: To support AI-specific workloads.

Traditional batch-only systems will not disappear, but they will be integrated into more dynamic architectures.

5.3 Architectural Principles for the Next 5–10 Years

The most important architectural principles will likely be:

- Incrementality by default
Systems must assume continuous updates.
- Separation of compute and storage
To scale independently and optimize costs.

- Log-based architectures
Event logs as the source of truth.
- Multi-model indexing
Support for relational, columnar, and vector indexes.
- Observability and reproducibility
Especially for AI training and evaluation pipelines.
- Hybrid consistency models
Balancing low latency with correctness.

AI systems will not replace data systems. Instead, they will make data infrastructure more central and more complex.

6. Conclusion

Modern data processing architectures have evolved from separate domains: transactional databases, analytical warehouses, streaming systems, and CDC pipelines. Originally, they solved different problems. Today, AI workloads connect them.

AI amplifies the need for:

- Incremental computation
- Continuous ingestion
- High read scalability
- Fresh and consistent derived data

I argue that the future will not be a single universal system, but a tightly integrated ecosystem built around streaming backbones, incremental processing, and scalable serving layers. Systems that can efficiently maintain derived state under continuous change will be at the core of future AI infrastructure.

In the next 5–10 years, the ability to combine analytical rigor, real-time responsiveness, and AI-specific indexing (such as vector search) will define competitive data platforms. The systems that succeed will not just store data, but continuously maintain knowledge in an efficient and scalable way.